Serial Python Minimalis: Tutorial #06-Struktur Data Dasar

Ahmad R. T. Nugraha

ver. 21 April 2026

Selain tipe data dasar seperti `int`, `float`, `bool`, `str`, dan sebagainya, Python menyediakan berbagai tipe data majemuk (*compound data types*), yaitu tipe data yang digunakan untuk mengelompokkan objek-objek. Struktur `list`, yang telah diperkenalkan sebelumnya, merupakan contoh paling sederhana dari tipe data majemuk. Struktur `list` akan dibahas lebih jauh pada sesi kali ini bersama dengan beberapa struktur data lain yang sangat berguna untuk menyimpan objek. Kita akan bahas juga perbedaan penting antara objek yang dapat diubah (*mutable*) dan tidak dapat diubah (*immutable*).

1 List

Kita sudah pernah mengeksplorasi penggunaan dasar `list` dalam Python, seperti proses kreasinya, pengindeksan, pemotongan (*slicing*), penambahan (*appending*), dan perluasan (*extending*). Secara umum, `list` adalah urutan objek dari tipe apa pun yang terurut dan bersifat dapat diubah (*mutable*). Struktur `list` dikatakan terurut karena setiap elemennya dapat dirujuk menggunakan indeks bilangan bulat yang unik dan memiliki panjang yang dapat diketahui menggunakan fungsi bawaan `len`:

```
# Sebuah list dengan elemen yang diindeks pada lst[0], lst[1], lst[2],
↪  lst[3].
lst = ['A', 2.0, 'C', 9]
lst[2] # elemen ketiga dari list
```

Output:

```
'C'
```

```
len(lst)
```

Output:

```
4
```

`list` bersifat *mutable* yang berarti elemennya dapat diubah langsung di tempat (berbeda dengan string):

```
lst[2] = -1
lst
```

Output:

```
['A', 2.0, -1, 9]
```

Perubahan ini tidak menjadi masalah karena `lst[2]` sebelumnya merupakan referensi ke objek string `'C'`. Kemudian, kita cukup menetapkannya kembali ke objek baru, yaitu bilangan bulat `-1`. Namun, string itu sendiri bersifat tidak dapat diubah *immutable*:

```
my_str = "hello"
my_str
```

Output:

```
'hello'
```

```
my_str[1] = "a"
```

Output:

```
TypeError                                 Traceback (most recent call last)
Input In [x], in <module>
----> 1 my_str[1] = "a"

TypeError: 'str' object does not support item assignment
```

Dalam situasi di atas, satu-satunya solusi adalah mendefinisikan objek `str` yang sepenuhnya baru, contohnya:

```
my_str = my_str[0] + "a" + my_str[2:]
my_str
```

Output:

```
'hallo'
```

Meskipun objek *immutable* tampak kurang fleksibel, ada keunggulan dari sisi kecepatan dan keamanan. Objek ini juga dapat diteruskan ke berbagai fungsi maupun objek lain tanpa perlu disalin, karena isinya dijamin tidak akan berubah di masa mendatang.

Fakta bahwa objek *mutable* dapat berubah setelah dibuat akan membawa konsekuensi penting ketika objek yang sama ditetapkan pada lebih dari satu nama variabel. Contoh berikut akan memperjelas situasinya.

```
lst1 = ['a', 'b', 'c']
lst2 = lst1
```

```
lst1.append('d') # Menambahkan elemen ke lst1

lst1 # Sesuai perkiraan
```

Output:

```
['a', 'b', 'c', 'd']
```

```
lst2 # lst2 ternyata ikut berubah!
```

Output:

```
['a', 'b', 'c', 'd']
```

Apa yang sebenarnya terjadi di sini? Untuk memahaminya, cara terbaik adalah membayangkan nama variabel (seperti `lst1`) sebagai sebuah label yang menempel pada objek `list` yang mendasarinya, bukan sebagai objek itu sendiri. Pada kode di atas, objek list `['a', 'b', 'c']` dibuat di suatu tempat di dalam memori komputer. Kemudian, nama variabel `lst1` ditugaskan untuk merujuk pada objek tersebut. Penetapan `lst2 = lst1` membuat nama `lst2` menunjuk ke objek yang persis sama. Dengan kata lain, `lst1` dan `lst2` adalah referensi ke satu `list` yang sama. Pengaksesan `list` tersebut melalui salah satu nama variabel akan mengubah isinya.

Objek *immutable* seperti string dan angka tentu saja dapat ditugaskan ke banyak nama variabel, tetapi karena nilainya sama sekali tidak dapat diubah, tidak akan ada perilaku tak terduga yang terjadi. Variabel apa pun dapat ditugaskan ke objek yang baru; perhatikan contoh berikut:

```
n = 1
m = n
m = m + 1

n
```

Output:

```
1
```

```
m
```

Output:

```
2
```

Selama operasi ini berlangsung, `n` tetap menjadi referensi ke objek *immutable* `1`. Variabel `m` awalnya juga merujuk ke objek ini, tetapi penetapan `m = m + 1` menghasilkan objek baru, yaitu `2`, dan menugaskan kembali nama `m` kepadanya (dan sama sekali tidak mengubah angka `1` yang dirujuk oleh `n`). Python memiliki metode bawaan, `id()`, untuk

melacaknya. Biasanya, fungsi ini akan mengembalikan alamat objek di dalam memori komputer:

```
n = 1
id(n)
```

Output:

```
4400245040
```

```
m = n
id(m) # Objek yang sama, id yang sama
```

Output:

```
4400245040
```

```
m = m + 1
id(m) # Sekarang m adalah objek int baru dengan id yang berbeda
```

Output:

```
4400245072
```

```
id(n) # n tidak berubah
```

Output:

```
4400245040
```

Jika kita memang menginginkan salinan `list` yang saling bebas (*independent*), panggil metode `copy()`:

```
lst1 = ['a', 'b', 'c']
lst2 = lst1.copy() # alternatif lain, potong seutuhnya: lst1[:]

lst1.append('d')
lst1
```

Output:

```
['a', 'b', 'c', 'd']
```

```
lst2 # lst2 tidak berubah
```

Output:

```
['a', 'b', 'c']
```

```
id(lst1), id(lst2) # karena ia adalah objek yang berbeda dari lst1
```

Output:

```
(4456958976, 4445977280)
```

Sementara itu, untuk mendapatkan indeks suatu elemen di dalam `list`, panggil metode `index`:

```
lst2.index('c')
```

Output:

```
2
```

Struktur `list` dapat diurutkan langsung di tempat (*in-place*) menggunakan metode `sort` (yang tidak mengembalikan nilai apa pun):

```
masses = [44, 28, 16, 32, 4]
masses.sort()
masses
```

Output:

```
[4, 16, 28, 32, 44]
```

Sebagai alternatif, fungsi bawaan `sorted` akan mengembalikan objek `list` baru dengan urutan yang sudah disesuaikan, sementara `list` aslinya tetap tidak berubah:

```
masses = [44, 28, 16, 32, 4]
sorted(masses) # mengembalikan list terurut yang baru
```

Output:

```
[4, 16, 28, 32, 44]
```

```
masses # tidak berubah
```

Output:

```
[44, 28, 16, 32, 4]
```

Python akan menolak untuk mengurutkan sebuah `list` apabila elemen-elemennya tidak dapat dibandingkan nilainya secara logis:

```
lst1 = [1, 0, "a"]
lst1.sort()
```

Output:

```
TypeError                                 Traceback (most recent call last)
Input In [x], in <module>
----> 1 lst1.sort()
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

Karena tidak ada cara yang masuk akal untuk membandingkan nilai antara objek string dan bilangan bulat, Python memunculkan pengecualian `TypeError`.

2 Tuple

Struktur `tuple` adalah urutan objek terurut yang bersifat *immutable*. Seperti halnya `list`, `tuple` dapat menampung objek dari berbagai tipe. Struktur `tuple` didefinisikan dengan memberikan urutan elemen di dalam tanda kurung biasa atau dengan meneruskan objek yang sesuai (yaitu yang bersifat *iterable*) ke dalam konstruktor `tuple`:

```
tpl1 = (0, 'a', 2.3)
tpl1
```

Output:

```
(0, 'a', 2.3)
```

```
tpl2 = tuple('abcdef')
tpl2
```

Output:

```
('a', 'b', 'c', 'd', 'e', 'f')
```

Pada dasarnya, dalam banyak kasus, tanda kurung biasa tidak diperlukan dan urutan yang dipisahkan oleh koma sudah cukup:

```
tpl1 = 0, 'a', 2.3
tpl1
```

Output:

```
(0, 'a', 2.3)
```

Untuk membuat `tuple` dengan elemen tunggal, tanda koma di akhir sangat diperlukan:

```
a = (3)
a # ini adalah bilangan bulat 3
```

Output:

```
3
```

```
a = (3,)
a # sebuah tuple dengan panjang 1 (sebuah "singleton")
```

Output:

```
(3,)
```

Sebagai struktur data yang *immutable*, penetapan nilai baru ke dalam elemen `tuple` tidak diperbolehkan, begitu pula penambahan maupun penghapusan elemen tidak didukung oleh struktur ini:

```
tpl2[0] = "z"
```

Output:

```
TypeError                                 Traceback (most recent call last)
Input In [x], in <module>
----> 1 tpl2[0] = "z"

TypeError: 'tuple' object does not support item assignment
```

3 Himpunan (`set`)

Himpunan atau `set` dalam Python adalah sekumpulan objek tak terurut (*unordered*) yang bersifat *mutable* dan tidak memiliki elemen duplikat:

```
s = set([1, 1, 'a', 'b', 1, 2, 'a', 1.0])
s
```

Output:

```
{1, 2, 'a', 'b'}
```

Perhatikan bahwa `1` dan `1.0` dianggap sebagai duplikat meskipun keduanya merupakan objek dengan tipe yang berbeda (yakni `int` dan `float`) karena secara matematis nilainya setara. Sebuah `set` yang tidak kosong juga dapat didefinisikan dengan meletakkan elemen-elemennya secara langsung di dalam tanda kurung kurawal:

```
s = {1, 1, 'a', 'b', 1, 2, 'a', 1.0}
```

Sebagai catatan, `s = {}` mendefinisikan sebuah *dictionary* (akan kita bahas belakangan), bukan sebuah `set` kosong. Karena tidak memiliki urutan yang baku, `set` tidak dapat diakses melalui indeks:

```
s[0]
```

Output:

```
TypeError                                 Traceback (most recent call last)
Input In [x], in <module>
----> 1 s[0]
```

```
TypeError: 'set' object is not subscriptable
```

Sebagai objek yang bersifat *mutable*, `set` memiliki berbagai metode untuk menambah dan menghapus elemen:

```
s.add(3) # menambah satu elemen
s
```

Output:

```
{1, 2, 3, 'a', 'b'}
```

```
s.update([2, 3, 4]) # menambah banyak elemen sekaligus dari sebuah koleksi
s
```

Output:

```
{1, 2, 3, 4, 'a', 'b'}
```

```
s.remove(4) # menghapus elemen tertentu
s
```

Output:

```
{1, 2, 3, 'a', 'b'}
```

```
s.pop() # menghapus dan mengembalikan sembarang elemen
```

Output:

```
1
```

```
s
```

Output:

```
{2, 3, 'a', 'b'}
```

Struktur `set` juga mendukung penggunaan operator matematis berikut: `-` (selisih / *difference*), `^` (selisih simetris / *symmetric difference*), `&` (irisan / *intersection*), dan `|` (gabungan / *union*):

```
s1 = {"A", "B", "C", "D"}
s2 = {"C", "D", "X", "Y", "Z"}

s1 - s2 # elemen yang ada di s1 tetapi tidak ada di s2
```

Output:

```
{'A', 'B'}
```

```
s2 - s1 # elemen yang ada di s2 tetapi tidak ada di s1
```

Output:

```
{'X', 'Y', 'Z'}
```

```
s1 ^ s2 # selisih simetris: elemen yang tidak dimiliki bersama oleh kedua set
```

Output:

```
{'A', 'B', 'X', 'Y', 'Z'}
```

```
s1 & s2 # irisan: elemen yang sama-sama dimiliki oleh kedua set
```

Output:

```
{'C', 'D'}
```

```
s1 | s2 # gabungan: seluruh elemen unik dari kedua set
```

Output:

```
{'A', 'B', 'C', 'D', 'X', 'Y', 'Z'}
```

4 Kamus (*dictionary*)

Kamus atau kita sebut *dictionary* dalam Python merupakan salah satu jenis *associative array*, yaitu sebuah struktur data yang memetakan objek kunci (*key*) ke objek nilai (*value*). Alih-alih menggunakan indeks berupa bilangan bulat, setiap nilai diasosiasikan dengan kunci unik (kuncinya wajib berupa objek yang *immutable*).

Cara paling sederhana untuk mendefinisikan *dictionary* adalah dengan menempatkan pasangan kunci-nilai (*key-value pairs*) sebagai urutan yang dipisahkan koma di dalam tanda kurung kurawal:

```
densities = {'Fe': 7.9, 'Al': 2.73, 'Pt': 21.4}
densities['Pt'] # dalam satuan g/cm3
```

Output:

```
21.4
```

Percobaan pengambilan nilai menggunakan kunci yang tidak ada di dalam *dictionary* akan memicu pengecualian `KeyError`:

```
densities["Ni"]
```

Output:

```
KeyError                                  Traceback (most recent call last)
Input In [x], in <module>
----> 1 densities["Ni"]

KeyError: 'Ni'
```

Metode get memungkinkan kita untuk memberikan nilai bawaan (*default*) untuk berjaga-jaga apabila kunci yang dicari ternyata tidak terdapat di dalam *dictionary*:

```
print(densities.get("Ni")) # mengembalikan None jika Ni tidak ada
```

Output:

```
None
```

```
print(densities.get("Ni", "No data available"))
```

Output:

```
No data available
```

Dictionary bersifat *mutable,* sehingga nilai yang sudah ada dapat diubah dan pasangan kunci-nilai baru dapat disisipkan sewaktu-waktu:

```
densities['Fe'] = 7.85
densities['Cu'] = 8.94
densities.update({'Hg': 13.6, 'Au': 19.3})

densities
```

Output:

```
{'Fe': 7.85, 'Al': 2.73, 'Pt': 21.4, 'Cu': 8.94, 'Hg': 13.6, 'Au': 19.3}
```

Iterasi suatu *dictionary* akan mengembalikan kunci-kuncinya secara berurutan sesuai dengan waktu saat kunci tersebut dimasukkan:

```
for element in densities:
    print(element)
```

Output:

```
Fe
Al
Pt
Cu
Hg
Au
```

Kunci-kunci *dictionary* juga dapat diekstrak secara lebih eksplisit melalui pemanggilan `densities.keys()`. Sementara itu, untuk mengakses nilainya saja, gunakan `densities.values()`:

```
for density in densities.values():
    print(density)
```

Output:

```
7.85
2.73
21.4
8.94
13.6
19.3
```

Sering kali kita ingin mengiterasi kunci sekaligus nilainya bersamaan, yang dapat diakomodasi oleh metode `densities.items()`:

```
for element, density in densities.items():
    print(element, density)
```

Output:

```
Fe 7.85
Al 2.73
Pt 21.4
Cu 8.94
Hg 13.6
Au 19.3
```

Terakhir, perlu diperhatikan bahwa sebuah *dictionary* kosong dapat dibuat cukup dengan sepasang tanda kurung kurawal tanpa isi:

```
my_dict = {}
```

Sekali lagi, ingat bahwa struktur ini bukanlah representasi dari `set` kosong.

Studi Kasus

Kasus 1 *Dictionary* sebagai Basis Data Sederhana

Salah satu penerapan *dictionary* adalah sebagai basis data sederhana. Nilai-nilai dapat disimpan berpasangan dengan kunci (*keys*) daripada ditugaskan ke nama variabel secara individu. Cara ini jauh lebih mudah untuk mengelola data dengan keuntungan kuncinya dapat berupa string sembarang (atau tipe data *immutable* lainnya), sedangkan nama variabel dibatasi oleh aturan sintaksis Python.
Tunjukkan bagaimana cara menggunakan *dictionary* bersarang (*nested dictionary*) untuk menyimpan, mengakses, memperbarui, dan menggabungkan data sifat-sifat fisik dari beberapa unsur Golongan IV (seperti C, Si, Ge, Sn, dan Pb).

Solusi

Untuk menyimpan beberapa sifat dari unsur C, Si, dan Ge, kita dapat menyusun sebuah *dictionary* bersarang dengan cara berikut:

```
element_properties = {
    'C': {'mass /u': 12.0, 'Tm /K': 3823, 'Tb /K': 5100,
          'rho /g.cm-3': 3.51, 'IE /eV': 11.26, 'atomic radius /pm':
          ↪  77.2},
    'Si': {'mass /u': 28.1, 'Tm /K': 1683, 'Tb /K': 2628,
           'rho /g.cm-3': 2.33, 'IE /eV': 8.15, 'atomic radius /pm':
           ↪  117},
    'Ge': {'mass /u': 72.6, 'Tm /K': 1211, 'Tb /K': 3103,
           'rho /g.cm-3': 5.32, 'IE /eV': 7.90, 'atomic radius /pm':
           ↪  122.5}
}

element_properties['C']
```

Output:

```
{'mass /u': 12.0, 'Tm /K': 3823, 'Tb /K': 5100, 'rho /g.cm-3': 3.51,
 ↪  'IE /eV': 11.26, 'atomic radius /pm': 77.2}
```

Kita dapat mengakses data spesifik dengan memanggil kuncinya secara beruntun:

```
IE = element_properties['Ge']['IE /eV']
print(f'The ionization energy of Ge is {IE} eV.')
```

Output:

```
The ionization energy of Ge is 7.9 eV.
```

Kita juga bisa mengiterasi *dictionary* tersebut untuk mencetak massa jenis dari seluruh unsur yang tersimpan:

```
print('Densities of Group IV elements (g.cm-3)')
for symbol, properties in element_properties.items():
    density = properties['rho /g.cm-3']
    print(f'{symbol:>2s}: {density:5.2f}')
```

Output:

```
Densities of Group IV elements (g.cm-3)
 C:  3.51
Si:  2.33
Ge:  5.32
```

Sebagai struktur data yang *mutable*, *dictionary* dapat diperbarui. Misalnya, kita dapat menambahkan entri baru yang bersesuaian dengan titik leleh dalam derajat Celcius untuk setiap unsur:

```
for symbol, properties in element_properties.items():
    Tm_K = properties['Tm /K']
    element_properties[symbol]['Tm /degC'] = Tm_K - 273

print(f"Melting point of silicon: {element_properties['Si']['Tm
 ↪  /degC']} degC")
```

Output:

```
Melting point of silicon: 1410 degC
```

Sejak versi Python 3.9, dua buah *dictionary* dapat digabungkan secara langsung menggunakan operator |:

```
more_element_properties = {
    'Sn': {'mass /u': 118.7, 'Tm /K': 505, 'Tb /K': 2543,
           'rho /g.cm-3': 7.29, 'IE /eV': 7.34, 'atomic radius /pm':
            ↪  140.5},
    'Pb': {'mass /u': 207.2, 'Tm /K': 601, 'Tb /K': 2013,
           'rho /g.cm-3': 11.3, 'IE /eV': 7.42, 'atomic radius /pm':
            ↪  175}
}

# Tambahkan sifat unsur Sn dan Pb ke dalam dictionary awal.
element_properties = element_properties | more_element_properties
print(element_properties.keys())
```

Output:

```
dict_keys(['C', 'Si', 'Ge', 'Sn', 'Pb'])
```

Sebagai catatan, pada praktiknya, data dengan struktur seperti ini mungkin akan lebih baik jika disimpan ke dalam sebuah DataFrame menggunakan modul `pandas`, salah satu pustaka yang cukup banyak digunakan praktisi sains data.

Kasus 2 Penguraian Rumus Kimia

Tuliskan sebuah fungsi untuk mengurai (*parse*) rumus kimia dengan format `'CH3CH2Cl'` menjadi suatu `list` yang berisi simbol-simbol unsur di dalamnya sesuai dengan urutan kemunculannya (contohnya, `list` berupa `['C', 'H', 'C', 'H', 'Cl']`).
Gunakan fungsi bawaan `set` di Python untuk menentukan simbol unsur unik yang ada (dalam hal ini, `{'C', 'H', 'Cl'}`) dan kemudian kategorikan apakah rumus tersebut merupakan senyawa hidrokarbon (hanya mengandung karbon dan hidrogen) atau bukan.

Solusi

Fungsi berikut ini bertugas memeriksa simbol unsur di dalam string rumus dan mengembalikannya dalam bentuk `list`.

```python
def get_element_symbols(formula):
    """Mengembalikan semua simbol unsur di dalam string formula."""
    n = len(formula)
    i = 0
    symbols = []

    # Iterasi melalui string, perhatikan huruf kapital yang
    # menandakan awal dari sebuah simbol unsur.
    while i < n:
        if formula[i].isupper():
            # Sebuah simbol unsur baru
            symbols.append(formula[i])
        elif formula[i].islower():
            # Jika menemui huruf kecil, ia adalah karakter kedua
            # dari sebuah simbol unsur.
            symbols[-1] = symbols[-1] + formula[i]
        i += 1

    return symbols

# Uji fungsi tersebut.
formula = 'CH3CH2Cl'
symbols = get_element_symbols(formula)
print(formula, '->', symbols)
```

Output:

```
CH3CH2Cl -> ['C', 'H', 'C', 'H', 'Cl']
```

Untuk menentukan simbol unsur unik, kita tinggal membuat `set` dari `list` tersebut:

```
set(symbols)
```

Output:

```
{'C', 'Cl', 'H'}
```

```
# Pengujian lain.
formula = 'CH3CBr2CO2H'
symbols = get_element_symbols(formula)

print(formula, '->', symbols)
print('Unique symbols:', set(symbols))
```

Output:

```
CH3CBr2CO2H -> ['C', 'H', 'C', 'Br', 'C', 'O', 'H']
Unique symbols: {'Br', 'C', 'H', 'O'}
```

Dengan pendekatan ini, kita dapat mendefinisikan sebuah fungsi khusus untuk memastikan apakah suatu rumus kimia tergolong senyawa hidrokarbon atau tidak:

```
def is_hydrocarbon(formula):
    symbol_set = set(get_element_symbols(formula))
    hydrocarbon_set = {'C', 'H'}
    return hydrocarbon_set == symbol_set

is_hydrocarbon('CH3CH2Cl')
```

Output:

```
False
```

```
is_hydrocarbon('CH3CH3')
```

Output:

```
True
```

Kasus 3 Struktur Kisi Kristal

Mari kita tinjau struktur kisi kristal dari unsur-unsur logam transisi baris pertama:

- fcc: Cu, Co, Fe, Mn, Ni, Sc
- bcc: Cr, Fe, Mn, Ti, V
- hcp: Co, Ni, Sc, Ti, Zn

Beberapa unsur memiliki struktur kristal yang berbeda pada kondisi suhu dan tekanan yang berbeda. Gunakan `set` pada Python untuk mengelompokkannya dan tentukan logam mana saja yang:

(a) hanya eksis dalam struktur *face-centered cubic* (fcc), *body-centered cubic* (bcc), atau *hexagonal close-packed* (hcp);

(b) eksis dalam dua dari struktur-struktur ini;

(c) tidak membentuk struktur hcp.

Apakah ada di antara logam-logam tersebut yang eksis dalam ketiga struktur itu?

Solusi

Kita dapat menggunakan operasi himpunan (`set`) pada Python seperti gabungan (`|`), irisan (`&`), dan selisih (`-`) untuk menjawab pertanyaan-pertanyaan ini.

```python
# Definisikan himpunan untuk setiap struktur kristal
fcc = {'Cu', 'Co', 'Fe', 'Mn', 'Ni', 'Sc'}
bcc = {'Cr', 'Fe', 'Mn', 'Ti', 'V'}
hcp = {'Co', 'Ni', 'Sc', 'Ti', 'Zn'}

# Kumpulkan semua logam transisi baris pertama yang ada
all_metals = fcc | bcc | hcp

# (a) Logam yang HANYA eksis di satu struktur saja
only_fcc = fcc - bcc - hcp
only_bcc = bcc - fcc - hcp
only_hcp = hcp - fcc - bcc
only_one = only_fcc | only_bcc | only_hcp
print("(a) Hanya di satu struktur :", only_one)

# (b) Logam yang eksis tepat di dua struktur
fcc_bcc = (fcc & bcc) - hcp
fcc_hcp = (fcc & hcp) - bcc
bcc_hcp = (bcc & hcp) - fcc
in_two = fcc_bcc | fcc_hcp | bcc_hcp
print("(b) Eksis di dua struktur  :", in_two)

# (c) Logam yang tidak membentuk struktur hcp
not_hcp = all_metals - hcp
print("(c) Tidak membentuk hcp    :", not_hcp)

# Apakah ada yang eksis di ketiga struktur?
all_three = fcc & bcc & hcp
print("Eksis di ketiga struktur   :", all_three)
```

Output:

```
(a) Hanya di satu struktur : {'V', 'Cr', 'Zn', 'Cu'}
(b) Eksis di dua struktur  : {'Mn', 'Co', 'Sc', 'Ni', 'Ti', 'Fe'}
(c) Tidak membentuk hcp    : {'V', 'Fe', 'Mn', 'Cr', 'Cu'}
Eksis di ketiga struktur   : set()
```

Berdasarkan *output* di atas, kita dapat menyimpulkan bahwa tidak ada satu pun logam transisi baris pertama dari daftar tersebut yang memiliki ketiga struktur kisi kristal sekaligus karena hasilnya adalah `set()` yang berarti himpunan kosong.